

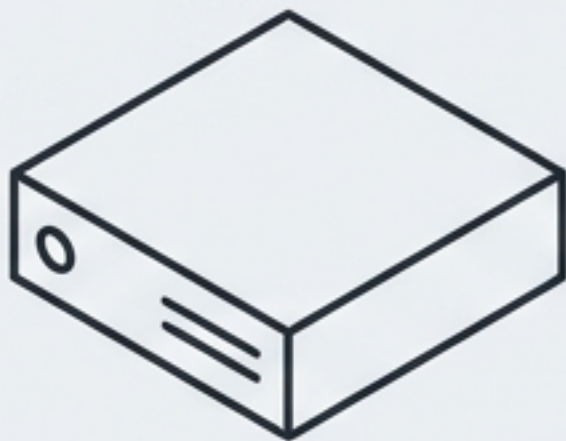


eCAMS Agent 보안 감사 보고서 (Final v3.0)

소스코드 정밀 분석 및 취약점 진단 (Source Code Audit & Vulnerability Assessment)

DATE:	2026-01-23
AUTHOR:	Prepared by Polaris
TARGET:	ecams_svr.c (1239 lines), lanapi.c, util.c, strcvt.c

**핵심 경고: AES 암호화 키 소스코드 하드코딩
및 원격 코드 실행(RCE) 취약점 식별**



Agent 개요

Socket 서버: 포트 29895 리스,
클라이언트 연결 시 Fork 방식
처리.

- 주요 기능: 파일 송수신(3-way handshake)
- TAR 압축/해제(Z/X 모드)
- 원격 명령 실행

핵심 위험 요소

Top 3 Critical Risks



1. AES 키 하드코딩 (Critical)

소스 유출 시 모든 암호화 통신 해독 및
서버 장악 가능.



2. RCE 취약점 (Critical)

system() 함수 직접 호출로 인한 무제한
셸 명령 실행 위험.



3. 가용성 위험 (Warning)

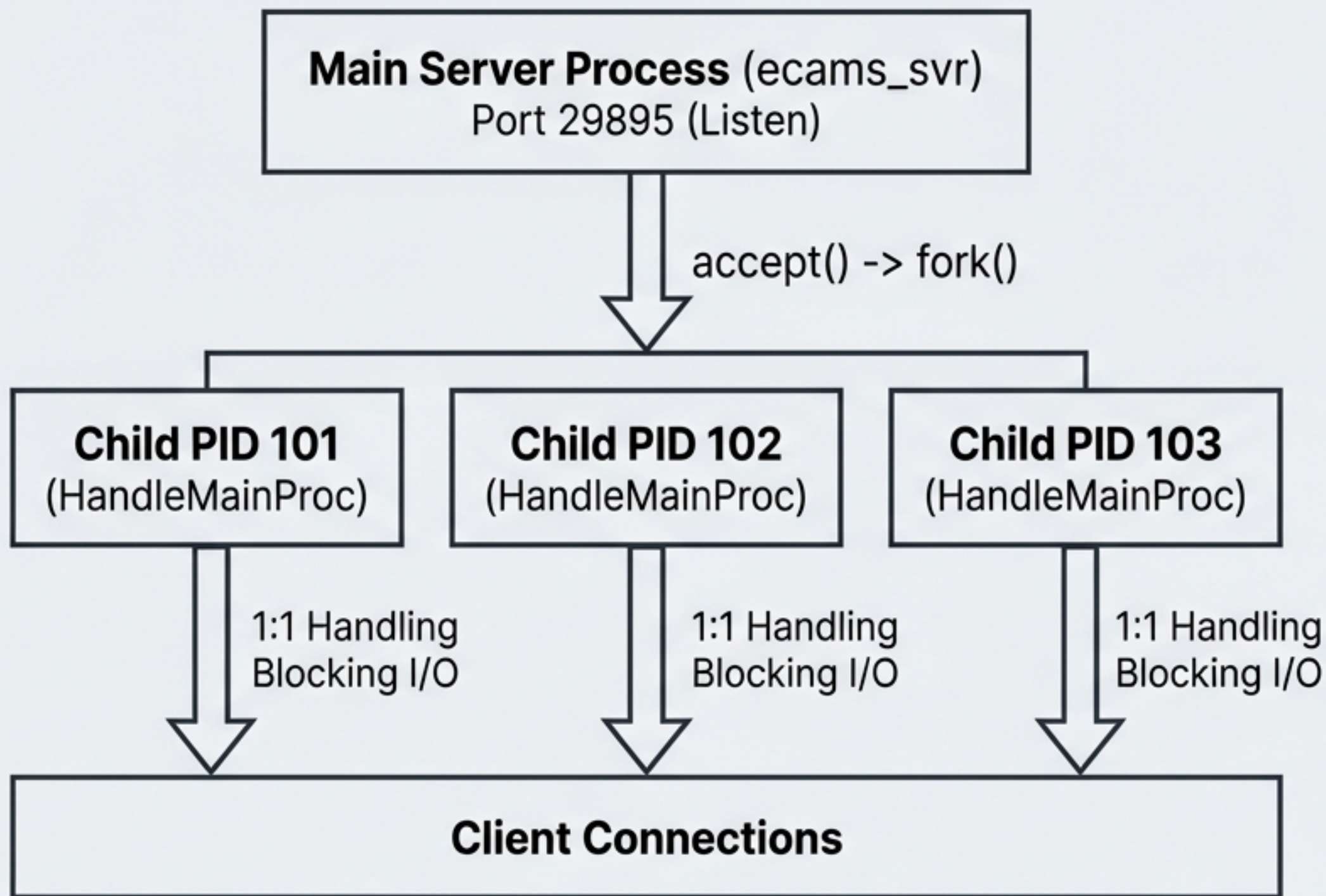
무제한 Fork 및 좀비 프로세스로 인한
서비스 거부(DoS) 가능성.



ACTION REQUIRED

즉시 외부 키 관리 시스템으로
전환 및 프로세스 제한 패치 적용
필수.

시스템 아키텍처 및 프로세스 흐름 (System Architecture)



상세 워크플로우

- **연결 처리:** 메인 서버는 `main()` (Line 365)에서 루프를 돌며 클라이언트 접속 시마다 자식 프로세스 생성.
- **파일 전송 프로토콜 (Chain):** 대용량 처리를 위한 First Chain -> Middle Chain -> Last Chain 방식의 3단계 전송 구조.
- **I/O 처리:** `lanapi.c`의 `Rcv_Data_Sock` 및 `SendDataToSock`을 통한 블로킹 방식 통신.

기능 명세 및 커맨드 매트릭스 (Functional Map)

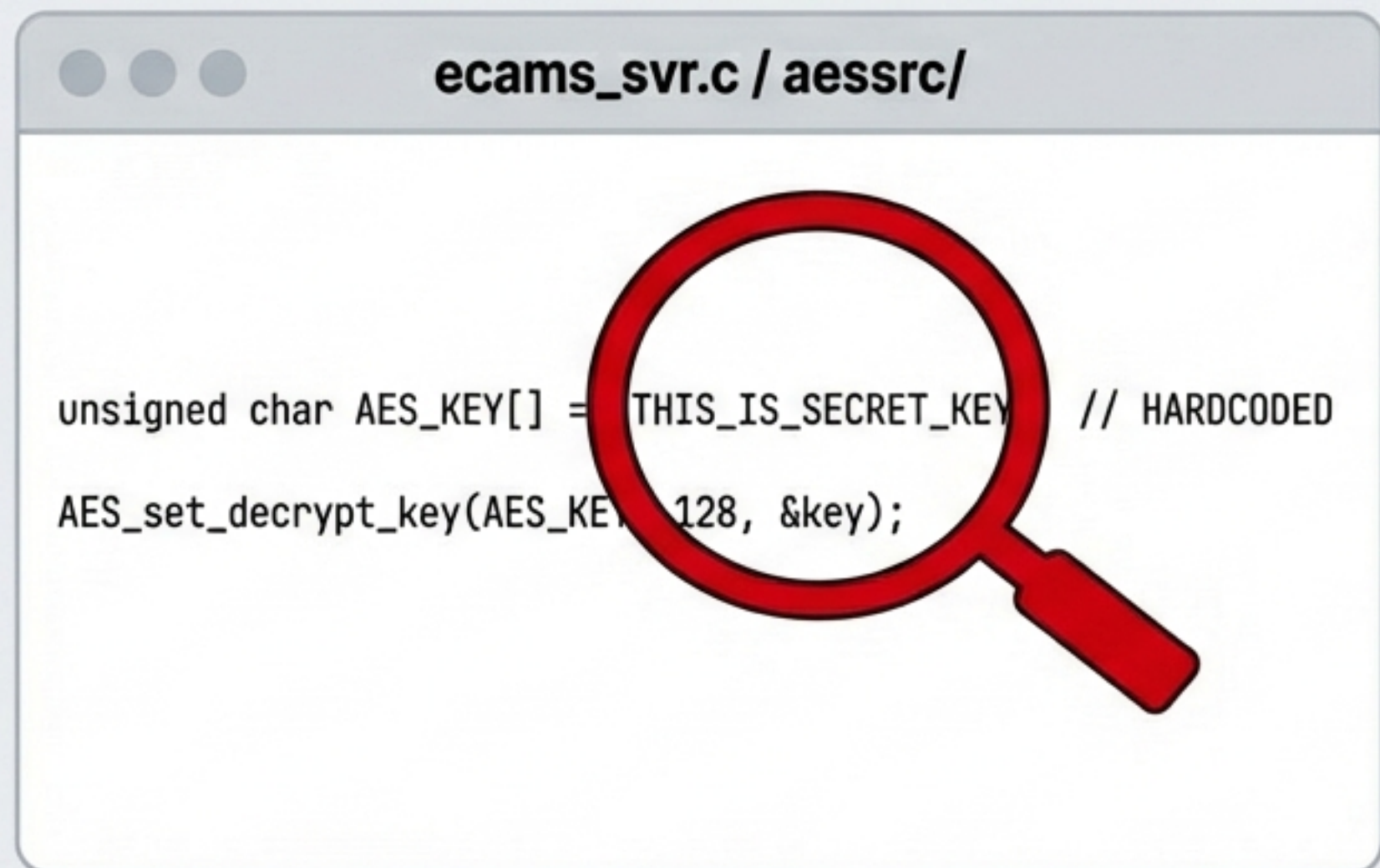
Command Code	Function Name	Line Location	Description / Mode
CmdSendTrx	ProcFileRcv / ProcFileSnd	Line 550	파일 전송: 모드 F(수신), G(송신), Z(해제), X(압축)
CmdSystem	ProcSystemCmd	Line 593	명령 실행: AES 암호화된 명령 수신 및 실행
CmdDECrypt	ProcDECrypt	Line 597	복호화: 파일 복호화 후 결과 반환
CmdEnvGet	ProcEnvGet	Line 605	환경변수: 서버 환경변수 조회 (DB 정보 노출 위험)



Insight: TAR 처리 로직(tarsrc/)은 Z모드(해제)와 X모드(압축+인덱스)를 자동으로 판별하여 처리함.

● CRITICAL RISK #1: 하드코딩된 암호화 키 (Hardcoded AES Key)

Code Evidence (Visual Block)



```
ecams_svr.c / aessrc/  
  
unsigned char AES_KEY[] = THIS_IS_SECRET_KEY // HARDCODED  
AES_set_decrypt_key(AES_KEY, 128, &key);
```

AES 암호화 키가 소스코드 내부에 평문으로 포함되어 있음.

공격 시나리오 (Attack Scenario)

Step 1. 공격자가 바이너리 또는 소스코드를 입수.

Step 2. 하드코딩된 키 추출 (Reverse Engineering 불필요).

Step 3. aes_128_cbc_decrypt 로직을 모방하여 네트워크 패킷 감청 및 조작.

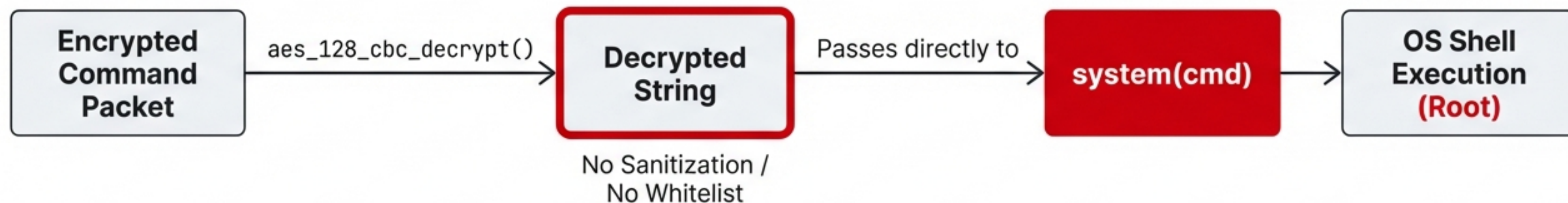
결과: 서버 내 모든 암호화 통신 무력화 및 원격 명령 주입 가능.

RISK: 심각 (Critical) - 즉시 수정 필요

● CRITICAL RISK #2: 원격 코드 실행 (RCE)

Focus Area: ProcSystemCmd Function (Line 621-697)

The RCE Pipeline (Horizontal Flow)



The Danger (위험성)

- 화이트리스트(허용 목록) 부재로 인해 공격자는 다음과 같은 악성 명령어 실행 가능:

Code Card

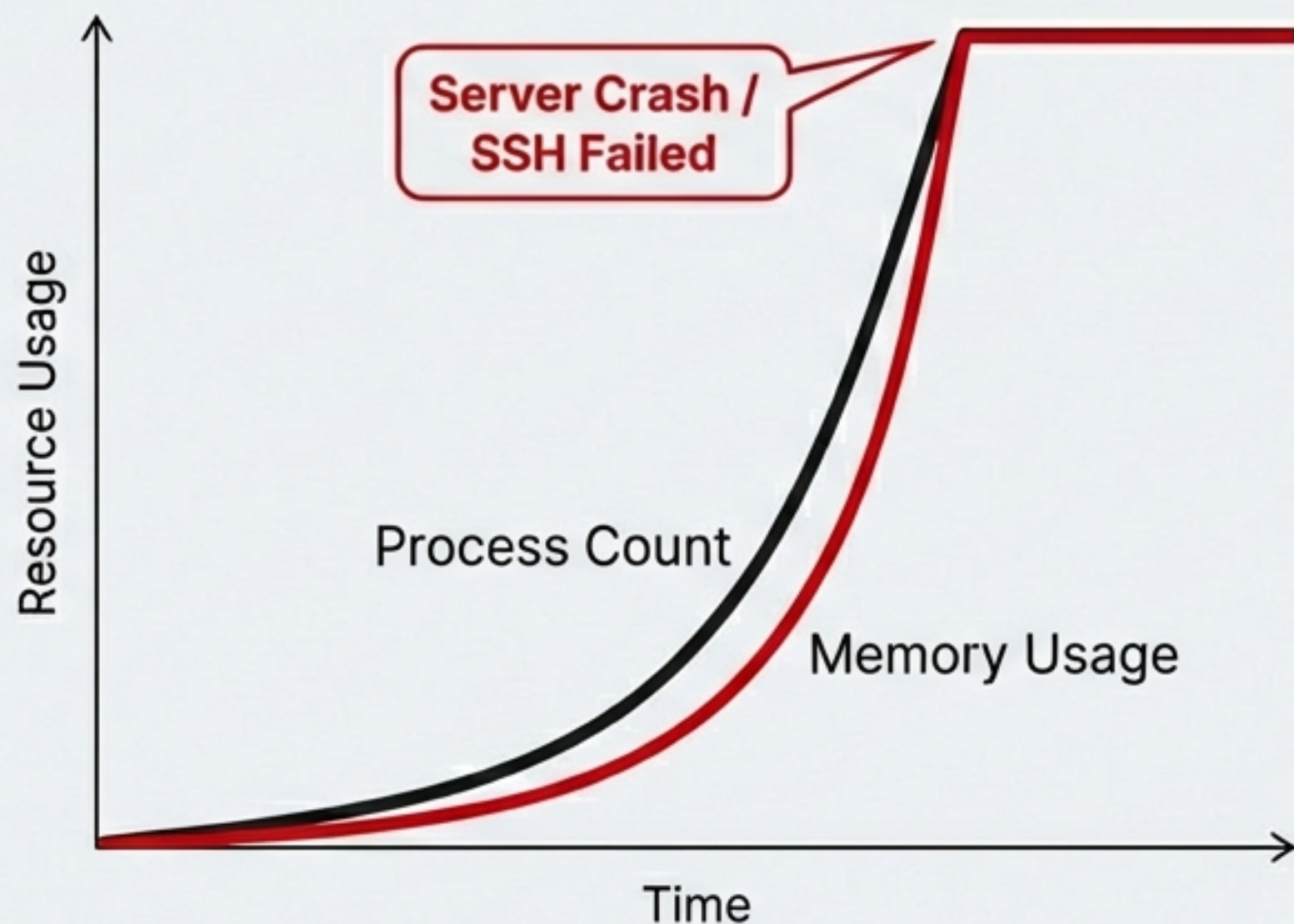
```
rm -rf / or nc -e /bin/sh
```

- Agent가 root 권한으로 실행될 경우 시스템 전체 장악(Root Compromise)으로 직결.

! OPERATIONAL RISK: Fork 폭탄 및 좀비 프로세스

Focus Area: Target: main() (Line 365) & Signal Handling

Resource Exhaustion Graph



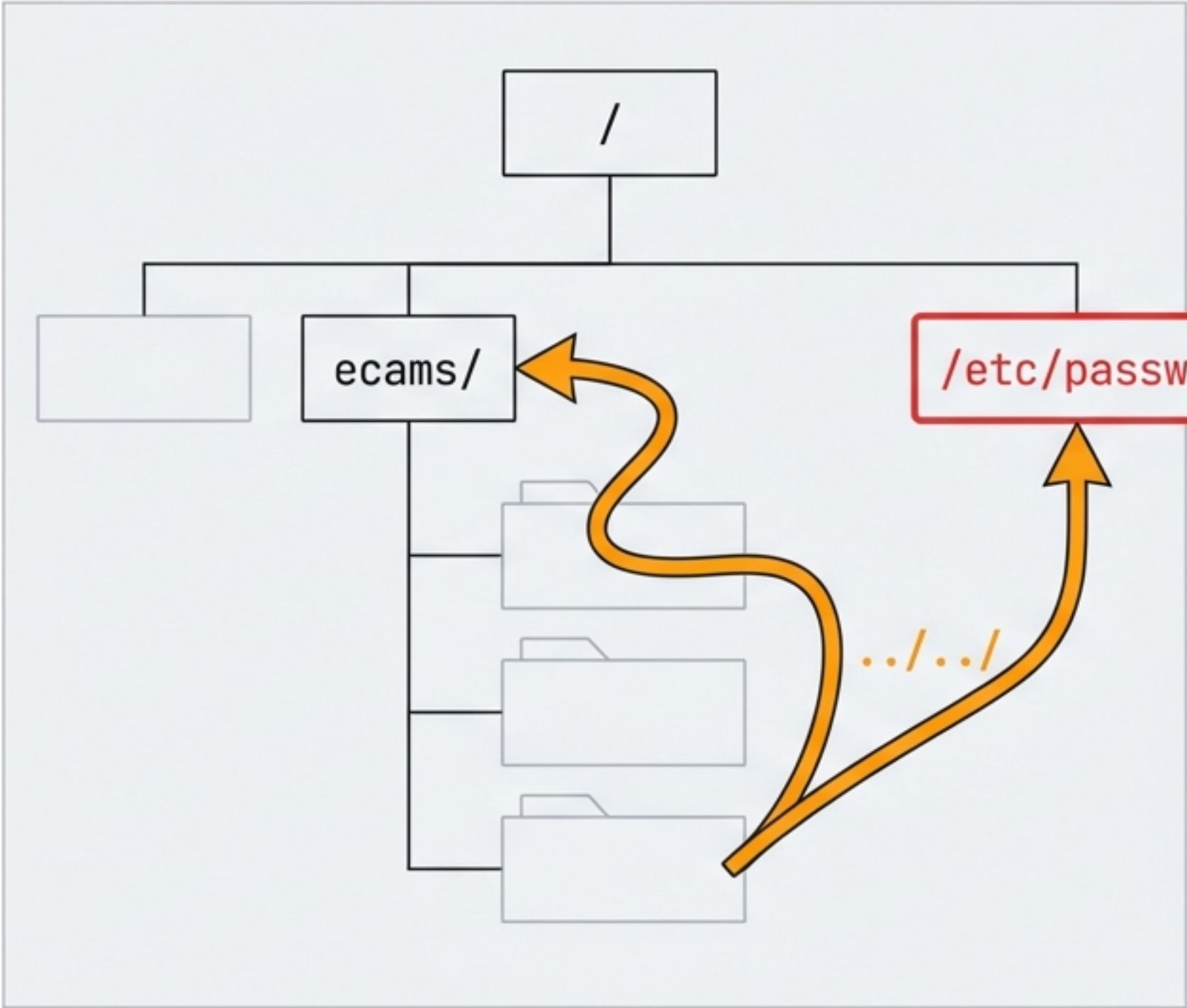
Analysis Details

- **Issue 1: 무제한 프로세스 생성 (Infinite Fork)**
 - MaxProcess 제한 로직이 존재하지 않음.
 - 동시 접속 폭주 시 프로세스 테이블 고갈(Resource Exhaustion) 발생 -> "fork() failed" 에러.
- **Issue 2: 좀비 프로세스 누적 (Zombie Accumulation)**
 - SIGCHLD 시그널 핸들러에서 wait() 처리가 미비함.
 - 자식 프로세스 종료 후에도 리소스가 반환되지 않아 시스템 메모리 점유.

Impact: 서비스 거부(DoS) 및 시스템 가용성 저하.

! SECURITY RISK: 파일 시스템 취약점

Directory Tree Illustration



Vulnerability List (Structured Layout)

!	1. 경로 탐색 (Path Traversal)
	위치: ecams_svr.c Line 960 (ProcFileRcv) 문제: 파일명에 대한 '.../' (상위 디렉토리 이동) 검증 로직 부재. 시스템 주요 파일 덮어쓰기 공격 가능.
!	2. 과도한 파일 권한
	문제: 파일 생성 시 권한을 0666 (모든 사용자 읽기/쓰기 가능)으로 설정. 기밀성 상실.
!	3. 환경변수 노출
	위치: ProcEnvGet (Line 805) 문제: DBUSER, DBPASS 등 민감한 환경변수를 평문으로 조회하여 전송.

소스코드 리스크 히트맵 (Source Code Responsibility Matrix)

● High Risk (즉시 수정)	
ProcSystemCmd (621-697)	RCE 원인, 화이트리스트 적용 필수.
ProcDECrypt (704-761)	임시파일 충돌 및 실행 위험.
● Medium Risk (개선 필요)	
main / HandleMainProc	Fork 폭탄 및 타임아웃 부재.
ProcFileRcv (889-1012)	권한 체크 및 경로 검증 추가.
ProcEnvGet	민감 정보 필터링 필요.
● Low Risk (유지)	
ProcFileSnd, GetFileInf	단순 조회 기능. 안전함.

조치 방안 1: 보안 취약점 패치 (Security Patching)

❗ 1. AES 키 외부화 (Target: ecams_svr.c Line 630)

Action

- 소스코드 내 키 삭제 및 외부 주입 방식으로 변경.

Code Implementation (Snippet)

Code Card

```
// Before:
char key[] = "hardcoded";

// After:
char *key = getenv("AES_SECRET_KEY");
if (!key) exit(1);
```

❗ 2. RCE 방지 (Target: ProcSystemCmd)

Action

- system() 호출 전 명령어 검증 로직 추가.

Method

- 허용된 명령어(예: tar, ls) 리스트와 대조 후 실행. 절대 경로 사용(execve) 권장.

조치 방안 2: 운영 안정성 확보 (Operational Stability)

! 1. 프로세스 제한 (Connection Limit)

Context: Code Target: Line 465 (fork 호출 전 검사)

Fix: main() 함수 도입부에 동시 실행 프로세스 수 카운터 추가 (예: Max 100).

! 2. 좀비 프로세스 제거 (Zombie Process Elimination)

Context: Signal Handler

Fix: SIGCHLD 핸들러 내에 `waitpid(-1, NULL, WNOHANG)` 루프 추가.

Code Card

```
while(waitpid(-1, NULL, WNOHANG) > 0);
```

! 3. Socket 타임아웃 (Socket Timeout)

Context: lanapi.c / Rcv_Data_Sock

Fix: `setsockopt(SO_RCVTIMEO)` 설정 (기본 5분)하여 네트워크 Hang 방지.

운영 및 로깅 개선 (Logging & Maintenance)

로그 시스템 현황 및 문제점

- ❌ 로그 레벨(INFO/ERROR) 구분 없음.
- ❌ 로그 로테이션(Rotation) 및 자동 삭제 정책 부재 (디스크 풀 위험).
- ❌ 중요 이벤트(Fork 실패, 명령 실행 실패 등) 누락.

개선 권고 사항

- 구조화된 로깅:**
 - 형식 통일: [TIMESTAMP] [PID] [LEVEL] Message
- 운영 스크립트:**
 - Systemd 서비스 파일 등록 및 모니터링 스크립트 작성.
- Log Rotate:**
 - logrotate 데몬 연동 설정 (일별 압축 및 30일 보관).

최종 결론 및 로드맵 (Final Verdict & Roadmap)

 **Current Status:**
CRITICAL (Red)

 **Target Status:**
MANAGED (Green)

-  1. **[보안] AES 키 외부화:** 환경변수/파일 분리 (즉시 실행).
-  2. **[보안] 명령어 화이트리스트:** `system` 함수 보호.
-  3. **[운영] 최대 프로세스 수 제한:** Max 100 설정.
-  4. **[운영] Socket 타임아웃:** Network Hang 방지.
-  5. **[보안] 파일 경로 검증:** 로직 추가 (`../` 차단).

위 5가지 항목 조치 완료 시 재진단(Re-audit) 수행 예정.